

Estructura de Carpetas en un Proyecto Clean

En el ecosistema actual de desarrollo de software, la velocidad de entrega a menudo colisiona con la sostenibilidad del código, creando una deuda técnica que asfixia el crecimiento de las compañías. La organización de un proyecto no es, bajo ningún concepto, una decisión meramente estética o una preferencia personal sin consecuencias; es, en esencia, un **contrato social** que define la visibilidad de los límites del sistema y garantiza que la lógica de negocio permanezca protegida frente a la volatilidad de los frameworks. Adoptar una arquitectura opinionada no implica seguir dogmas ciegamente, sino aplicar una metodología basada en la experiencia para que el equipo hable un mismo lenguaje técnico y estratégico. Al establecer una jerarquía clara entre dominio, aplicación e infraestructura, las organizaciones dejan de construir software rígido para empezar a desarrollar activos digitales resilientes. Este enfoque permite que la **soberanía técnica** de la empresa no dependa de si una librería de terceros como React, Fastify o Prisma queda obsoleta, sino de cómo la lógica interna interactúa con estas herramientas a través de interfaces bien definidas. Dominar esta separación de responsabilidades permite reducir drásticamente el 'time-to-market' de nuevas funcionalidades, ya que el desarrollador no pierde tiempo descifrando dónde debe ubicar cada pieza de lógica; la estructura misma le dicta el camino. A ello se suma la capacidad de escalar desde prototipos mínimos hasta sistemas complejos de alta disponibilidad sin necesidad de refactorizaciones traumáticas. En última instancia, una estructura robusta es la que responde con agilidad a las necesidades cambiantes del mercado, permitiendo que el repositorio de código sea un reflejo fiel del modelo de negocio y no un laberinto de dependencias cruzadas. La implementación de este modelo prepara a los equipos para alcanzar una **ventaja competitiva** real: la capacidad de iterar sobre el producto con la confianza de que el núcleo del sistema es inmune a los cambios en los adaptadores externos o en la infraestructura de persistencia. Al entender que el dominio nunca debe importar nada ajeno a sí mismo, protegemos el conocimiento más valioso de la compañía, asegurando que la **inversión de dependencias** no sea solo un patrón de diseño, sino un pilar fundamental de la estrategia operativa de TI.

Modelos de Implementación Estratégica

Estructura Mínima para Prototipado y Agilidad

Para equipos que inician un proyecto o validan una idea de negocio, la simplicidad es una virtud. Una estructura mínima que conserve las carpetas de dominio, aplicación e infraestructura permite establecer las bases de la Clean Architecture sin la fricción inicial de una granularidad excesiva. Este modelo facilita la curva de aprendizaje y acelera el despliegue de MVPs (Productos Mínimos Viables).

Estructura Escalable para Grandes Ecosistemas

Cuando el proyecto escala, la granularidad se vuelve indispensable. En este escenario, la capa de infraestructura se desglosa en servicios de persistencia, mensajería, observabilidad y controladores HTTP. Aquí es donde cobra relevancia el 'Composition Root', un orquestador que gestiona la inyección de dependencias y asegura que los adaptadores técnicos se vinculen correctamente con los puertos definidos en la capa de aplicación.

Convenciones y Semántica Profesional

La legibilidad es una forma de cortesía profesional que reduce la carga cognitiva del equipo. El uso de Pascal Case para entidades y Value Objects, junto con sufijos técnicos claros para los adaptadores (como 'InMemoryOrderRepository'), transforma el código en una documentación viva. Los casos de uso deben ser nombrados como verbos de acción, reflejando fielmente las operaciones de negocio que el sistema permite ejecutar.

Observaciones Clave

- El dominio debe ser totalmente agnóstico: nunca debe importar elementos de la capa de aplicación o infraestructura.
- Los Data Transfer Objects (DTO) deben residir obligatoriamente en la capa de aplicación y ser serializables.
- Se debe evitar el uso de 'barrels' (ficheros index) que crucen capas de arquitectura, para prevenir efectos secundarios no deseados e importaciones circulares.
- La configuración y las variables de entorno deben gestionarse exclusivamente en la capa de infraestructura o en módulos de configuración específicos.
- Los adaptadores dependen de los puertos (interfaces), pero el puerto nunca debe conocer la implementación del adaptador.

Visión de Futuro en la Gestión de Activos Digitales

La adopción de estas arquitecturas no es un fin en sí mismo, sino un medio para garantizar que el software sea una herramienta de crecimiento y no un lastre. Al separar la lógica de negocio de los detalles de implementación, las empresas ganan la libertad de cambiar de proveedor de base de datos o de framework web con un riesgo mínimo. Este nivel de desacoplamiento es lo que separa a las empresas tecnológicas líderes de aquellas que se ven atrapadas en sistemas heredados imposibles de mantener. La estructura propuesta actúa como un seguro ante la incertidumbre tecnológica del futuro.